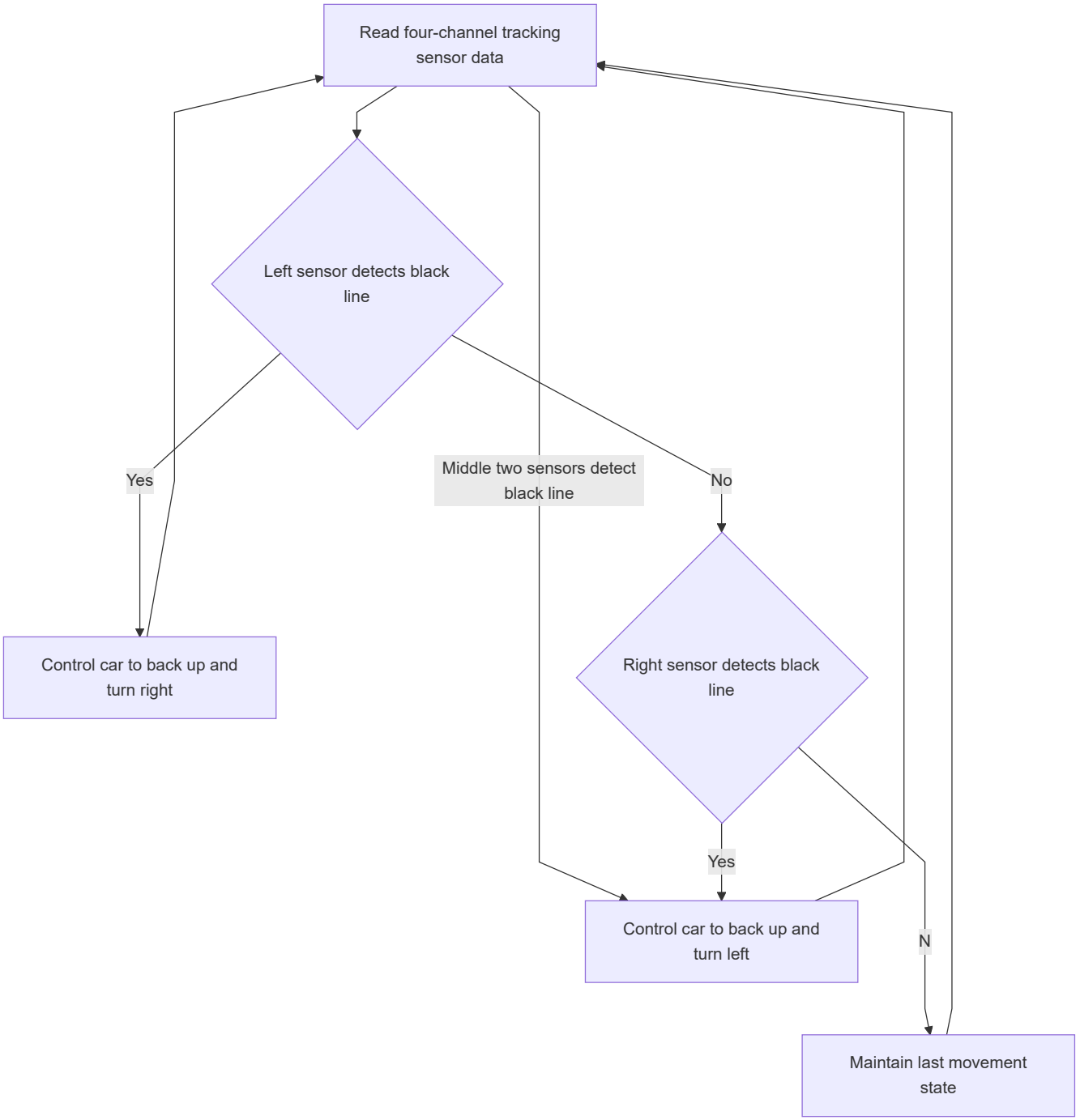
Box Yourself In

Function Description

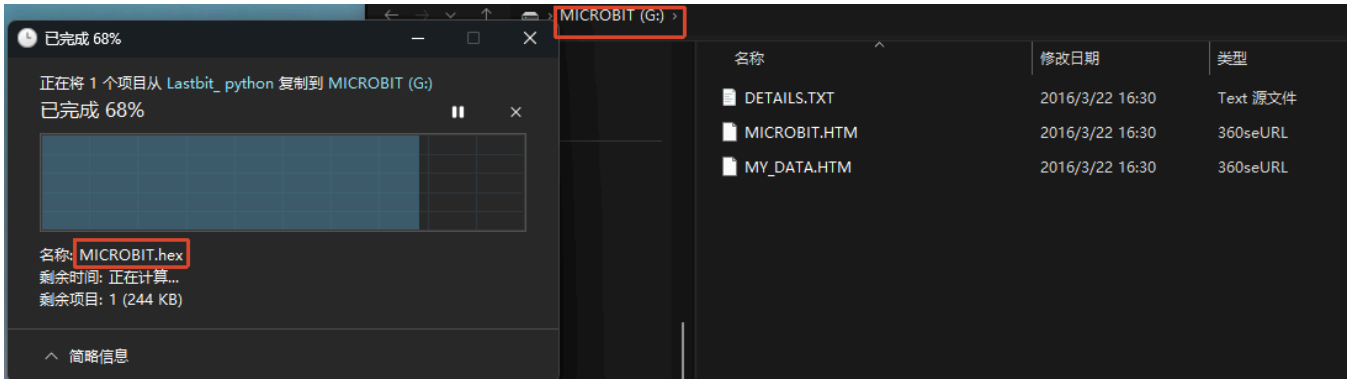
Earlier we learned how to read data from the four-channel line-tracking sensors. In this section, we will learn how to use that data to implement the "Stay Within the Boundary" function.

Working Principle



Experiment Steps

Make sure `LASTBIT.hex` is flashed before flashing, otherwise an error will occur and the module cannot be imported



1. Before flashing, switch the switch to the OFF position and use a microUSB data cable to connect the computer and the micro:bit main board. **Note that it is the Microbit interface, not the Charging port on the expansion board.**

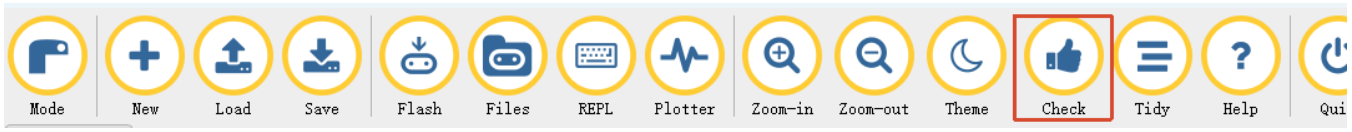
2. Open the Mu software. Here you can directly copy the program source code we provided; the operation steps are as follows. You can also enter the code manually in the editing window. Note! All English letters and symbols should be entered in English input mode. Use the Tab key for indentation, and the last line should end with a blank program.

   Click the `New` button in the toolbar at the top left, and a file titled "untitled" will appear.

3. Copy the content of the `Code Implementation` section provided below into the current file. You will notice a red dot after the title bar, indicating that the code content has been modified but is in an unsaved state.

Whether to save or not does not affect code checking or flashing.

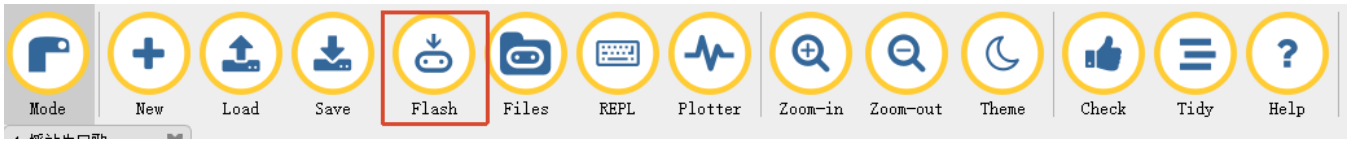
4. Now you can click `Check` to verify whether there are any problems with the code format.



5. Now if you have already connected the MICROBIT to the PC in advance following the steps above, and flashed `LASTBIT.hex`, the next step is to download the current program to the Microbit.

Click the `Flash` tool in the toolbar to start flashing!

During flashing, the orange-yellow indicator light near the Microbit main board will blink frequently. When the editor shows `Copied code onto micro:bit.` at the bottom, it means the flashing is complete, and the indicator light will no longer blink after flashing.



6. Unplug the microUSB data cable, place the car on the line-tracking track, and switch the switch to the ON position.

Code Implementation

```
# Stay Within the Boundary
from microbit import *
from lastbit import LASTBIT

Lastbit = LASTBIT()

base_speed = 60
display.show(Image.HAPPY)

def tracking_mode():
    # Read 4-channel tracking data
    track = Lastbit.read_track()
    if track is None:
        return
    for i in range(4):
        track[i] = 1 if track[i] == 0 else 0

    left = track[0]
    mid_l = track[1]
    mid_r = track[2]
    right = track[3]

    # Left side crossing the line
    if left == 1 or mid_l == 1:
        Lastbit.car_control('BACKWARD', base_speed, 500)
        Lastbit.car_control('SPINRIGHT', 150, 800)
        return

    # Right side crossing the line
    if mid_r == 1 or right == 1:
        Lastbit.car_control('BACKWARD', base_speed, 500)
        Lastbit.car_control('SPINLEFT', 150, 800)
        return

    # Both middle sensors crossing the line
    if mid_l == 1 and mid_r == 1:
        Lastbit.car_control('BACKWARD', base_speed, 1000)
        Lastbit.car_control('SPINLEFT', 150, 1000)
```

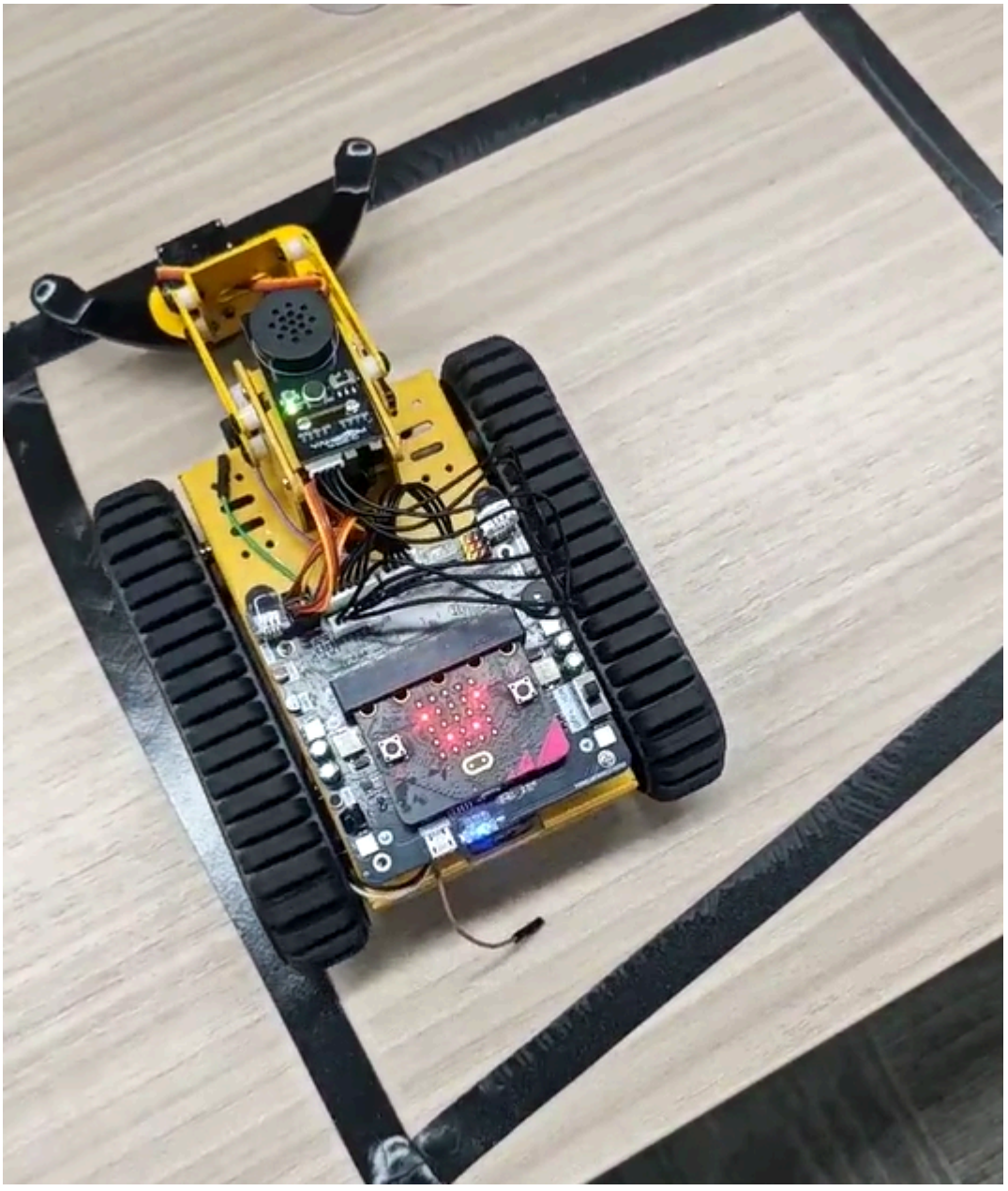
```
        return

    # No line crossed → go straight
    Lastbit.car_control('FORWARD', base_speed, 0)


while True:
    tracking_mode()
    sleep(10)
```

Experimental Phenomenon

Place the car inside a closed circular or square area enclosed with black tape. The car will move within the track. If it detects a black edge, it will back up and adjust its direction. The backward distance and turn angle can be adjusted through the time parameters.



Notes

Make sure the battery is fully charged and the four-channel tracking sensors have been adjusted to appropriate sensitivity via the potentiometers.

This case cannot use the accompanying map. You can stick black tape on the ground to form a closed circular or square area.